Die Messlatte, bevor das Rennen beginnt

Bevor eines der drei Vergleichsverfahren rechnen darf, klärt der Ordner vorpruefung/ drei Dinge: wie die Stadtteile für die Prüfung gemischt werden, welche einfachen Modelle geschlagen werden müssen und ob die Verfahren überhaupt zu den Daten passen. Dazu kommen Prüfungen, die zeigen, was mit diesen Daten erreichbar ist. Dieses Heft grenzt die Dateien voneinander ab und nennt jede Funktion mit ihrem Zweck.

Ordner: vorpruefung/
Umfang: 7 Dateien, 54 Funktionen
Anschluss an: Heft 1 und Band 1 und 2 der Laienfassung
Grundlage: der Quelltext im Abgabeordner, Stand 08.10.2026

Heft 1
prep/ und tests/

Heft 2
vorpruefung/

Heft 3
modelle/

TEIL 1

Der Ordner auf einen Blick

> Der Ordner heißt Vorprüfung, weil fast alles darin vor dem eigentlichen Verfahrensvergleich steht. Er legt die Messlatte und prüft, ob die Verfahren zu den Daten passen.

Fast alles in diesem Ordner läuft, bevor eines der drei Vergleichsverfahren angepasst wird. Die Dateien liefern einander zu: `v0_aufteilung.py` bestimmt, welche Stadtteile gemeinsam geprüft werden, `v1_baselines.py` rechnet auf dieser Aufteilung die Messlatte, und `v2_eignung.py` liest deren Werte. Eine Gegenprobe, eine Beschreibung der Aufteilung und eine nachträgliche Einordnung kommen hinzu.



Abbildung 1: Die Dateien des Ordners in der Reihenfolge des Ablaufs. `run.py` startet nur `v1_baselines.py` und `v2_eignung.py`, die übrigen laufen einzeln. `v4_decke.py` läuft nach `m03_struktur.py`, weil es dessen Ergebnisse einordnet.

DATEI	DIE FRAGE, DIE SIE BEANTWORTET	FUNKTIONEN
`run.py`	Startet Messlatte und Eignungsprüfung in einem Zug.	2
`v0_aufteilung.py`	Wie werden die Prüfgruppen zehnmal neu gemischt?	4
`panelprofil.py`	Wer sind die 30 Entwicklungs- und wer die 6 Hold-out-Stadtteile?	10
`v1_baselines.py`	Welche einfachen Modelle muss ein Verfahren schlagen?	8
`v2_eignung.py`	Passen die Verfahren zu den Daten?	13
`v3_spezifikation.py`	Hält die Krümmung auch bei unbekannten Stadtteilen?	6
`v4_decke.py`	Wie gut lässt sich die Einsatzart überhaupt vorhersagen?	11

TEIL 2

Wie sich die Dateien voneinander abgrenzen

> Die Dateien dieses Ordners hängen enger zusammen als die der Aufbereitung. Gerade deshalb lohnt der Blick auf die Grenzen: Wer liefert wem zu, und wer prüft wen?

Die Messlatte hat zwei Stufen

Der Begriff Baseline aus Band 1 ist in diesem Ordner zweistufig gebaut. Stufe 1 ist das stumpfeste denkbare Modell, das kein einziges Merkmal ansieht. Es beantwortet nur, ob in den Merkmalen überhaupt Information steckt. Stufe 2 ist das einfachste Modell, das zur Form der Daten passt: Es benutzt dieselben Merkmale wie die Verfahren, hat aber keine Einstellschraube. Erst wer Stufe 2 schlägt, hat gezeigt, dass sich der Mehraufwand lohnt.



Abbildung 2: Das Stufenmodell. Stufe 1 und Stufe 2 stehen beide in `v1_baselines.py`. Geschlagen werden muss Stufe 2, und die ist in den beiden Strängen ein jeweils anderes Modell.

Die Grundaufteilung und ihre Wiederholungen

Die Gruppenzuteilung steht seit Schritt 2 der Aufbereitung als Spalte in den Daten (Heft 1). `v0_aufteilung.py` verändert sie nicht, sondern mischt aus ihr zehn Wiederholungen. Das Hold-out bleibt dabei fest, und Wiederholung 0 ist genau die Zuteilung aus der Datei. `v0` schreibt keine eigene Datei, sondern liefert die Mischung an alle Skripte, die über zehn Wiederholungen rechnen. So vergleicht der spätere gepaarte Test Messlatte und Verfahren auf denselben Zeilen.

Messlatte und Vergleichsverfahren

`v1_baselines.py` rechnet nach demselben Protokoll wie später die Verfahren in Heft 3: dieselben zehn Wiederholungen, dieselben fünf Prüfgruppen, dieselben Zeilen. Die Messlatte ist damit ein Mitbewerber und kein bloßer Referenzwert. Der Unterschied zu Heft 3 liegt allein darin, dass hier nichts eingestellt wird, weil Stufe 2 keine Einstellschraube hat.

Eignung und Gegenprobe

`v2_eignung.py` prüft auf den Übungsstadtteilen der ersten Prüfgruppe, ob ein gerades Modell die Daten gut beschreibt. `v3_spezifikation.py` stellt die Folgefrage: Wird die Vorhersage für unbekannte Stadtteile besser, wenn man dem Modell Krümmung und Wechselwirkungen erlaubt? Die eine Datei misst also, wie gut

ein Modell die bekannten Stadtteile beschreibt, die andere, wie gut es unbekannte vorhersagt. Band 2, Teil 9 zeigt, dass beide Antworten hier auseinanderlaufen.

Beschreiben gegen Einordnen

`panelprofil.py` beschreibt nur, wie sich Entwicklungs- und Hold-out-Stadtteile zusammensetzen und wie die Prüfgruppen der Wiederholungen liegen, ohne ein einziges Modell. `v4_decke.py` rechnet dagegen Obergrenzen, an denen die Ergebnisse des Strukturstrangs gemessen werden. Zur Abgrenzung nach vorn: `deskriptiv.py` aus Heft 1 beschreibt, wie die Daten beschaffen sind. Ob ein Verfahren dazu passt, entscheidet erst `v2_eignung.py`.

WARUM DIE MESSLATTE VOR DEM RENNEN STEHT

Wer die Messlatte erst nach dem Ergebnis festlegt, kann sie unbewusst so legen, dass das Ergebnis gut aussieht. Deshalb laufen alle Dateien dieses Ordners außer `v4_decke.py` vor dem Verfahrensvergleich. Und auch in `v4_decke.py` hängen die beiden Obergrenzen von keinem Modellergebnis ab. Erst die Quote, wie viel davon ein Verfahren ausschöpft, liest die Ergebnisse aus Heft 3.

TEIL 3

Datei für Datei

> Für jede Datei: was sie tut, was sie liest und schreibt, und eine Tabelle mit jeder Funktion und ihrem Zweck. Die Reihenfolge folgt dem Ablauf.

SO LIEST MAN DIE TABELLEN

Eine Python-Datei ist eine Art Rezept. Eine *Funktion* ist ein einzelner, benannter Arbeitsschritt darin: Sie bekommt etwas hinein, tut etwas damit und gibt etwas zurück. Funktionen, deren Name mit einem Unterstrich beginnt (etwa `_md`), sind als interne Hilfsschritte gedacht. Einige davon leihen sich andere Dateien trotzdem aus. Die Funktion `main` oder `run` ist der *Startpunkt*: Sie läuft, wenn man die Datei aufruft, und ruft ihrerseits die übrigen Funktionen in der richtigen Reihenfolge auf.

In den folgenden Tabellen steht jede Funktion einzeln, in der Reihenfolge, in der sie im Quelltext vorkommt. Die Markierung *Startpunkt* zeigt, wo der Ablauf beginnt, *Hilfsfunktion* einen kleinen Zuarbeiter, *innen* eine Funktion, die nur innerhalb einer anderen Funktion existiert. Die Funktionszahlen in den Übersichten zählen solche inneren Funktionen mit.

---

vorpruefung/

run.py

Der eine Befehl der Vorprüfung

Startet Messlatte und Eignungsprüfung nacheinander.

LIEST	SCHREIBT
über v1 und v2 die beiden Analysedateien	über v1 und v2 deren Ergebnisdateien

Wie `build.py` in der Aufbereitung rechnet `run.py` selbst nichts. Es sorgt nur für die richtige Reihenfolge: erst die Messlatte, dann die Eignungsprüfung, weil diese die Werte der Messlatte liest. `v0`, `v3`, `v4` und `panelprofil` laufen einzeln.

FUNKTION	WOZU SIE DA IST
`schritt()` Hilfsfunktion	Schreibt die Überschrift eines Arbeitsschritts auf den Bildschirm.
`main()` Startpunkt	Startet erst `v1_baselines.py`, dann `v2_eignung.py`.

vorpruefung/

v0_aufteilung.py

Zehnmal neu gemischt

Erzeugt die zehn Wiederholungen der Gruppenzuteilung.

LIEST	SCHREIBT	BENUTZT VON
beide Analysedateien mit den Spalten `fold` und `ist_holdout`	keine Datei	v1, v3, panelprofil, m02, m03, m04, parametersensitivitaet, suchdiagnose

Bei nur 30 Entwicklungsstadtteilen hängt ein einzelnes Ergebnis stark davon ab, welche Stadtteile zufällig gemeinsam geprüft werden. Deshalb wird die ganze Bewertung zehnmal mit neu gemischten Gruppen wiederholt. Gemischt wird so, dass jede Gruppe sechs Stadtteile behält, Brandfälle enthält und das Hold-out unberührt bleibt.

Dabei wird nur festgelegt, welche Stadtteile gemeinsam geprüft werden. Kein Modell sieht dadurch eine Zeile mehr.

FUNKTION	WOZU SIE DA IST
`selten_je_stadtteil()`	Zählt je Stadtteil die Monate, in denen Brände überwiegen. Nach dieser Zahl wird bei der Gruppenbildung sortiert, damit jede Gruppe Brandfälle bekommt.
`wiederholte_aufteilung()`	Mischt die Prüfgruppen für eine Wiederholung neu. Gruppengröße, Sortierung nach Brandmonaten und Hold-out bleiben dabei erhalten. Für Wiederholung 0 wird geprüft, dass sie bitgenau der Zuteilung in der Datei entspricht.
`entwicklung_und_holdout()`	Trennt die 30 Entwicklungsstadtteile von den 6 Hold-out-Stadtteilen. Über diese Funktion holen sich `m02` und `m03` das Hold-out für die Schlussbewertung.
`_selbsttest()` Startpunkt	Prüft alle zehn Wiederholungen: Gruppengrößen, Brandfälle in jeder Gruppe, Hold-out unverändert, zehn verschiedene Mischungen, kein Stadtteil zugleich Übungs- und Prüffall. Läuft, wenn man die Datei direkt startet.

vorpruefung/

panelprofil.py

Wer sind die 30, wer die 6?

Beschreibt die Zusammensetzung von Entwicklungs- und Hold-out-Stadtteilen.

LIEST	SCHREIBT
beide Analysedateien und die Stadtteilgrenzen aus `data/raw/`	`results/panelprofil/`: fünf Tabellen und `panelprofil.md`

Eine Schlussbewertung auf sechs Stadtteilen ist nur lesbar, wenn man weiß, welche sechs das sind. Diese Datei hält es fest, und zwar unabhängig von jedem Modellergebnis. Sie ist rein beschreibend: kein Modell,

kein Test. Die Mischungen der Wiederholungen holt sie aus `v0_aufteilung.py`, mit festem Startwert, deshalb liefern zwei Läufe dieselbe Datei.

Eine Eigenschaft der Zuteilung wird dabei sichtbar: Weil die Stadtteile nach ihren Brandmonaten sortiert und dann reihum verteilt werden, landet der Stadtteil mit den meisten Brandmonaten zwangsläufig im Hold-out. Das ist kein Los, sondern eine Folge der Regel, und wird deshalb berichtet.

Zwei weitere Eigenschaften der Aufteilung stehen ebenfalls hier, weil sie von keinem Modell abhängen: wie viele Prüfstadtteile der späteren Wiederholungen schon bei der Suche nach den Einstellungen dabei waren, und wie viele Übungsstadtteile keine gemeinsame Grenze mit einem Prüfstadtteil haben.

FUNKTION	WOZU SIE DA IST
`lade()`	Liest beide Tabellen und prüft, dass sie dasselbe Hold-out tragen. Weichen sie ab, ist die Aufbereitung neu zu starten. Diese Datei repariert das nicht still.
`stadtteile()`	Eine Zeile je Stadtteil: Gruppe, Größe, Einsatzlast und häufigste Einsatzart. Sortiert wie die Zuteilungsregel, damit man sieht, warum welcher Stadtteil im Hold-out steht.
`klassenverteilung()`	Anteile der vier Einsatzarten, getrennt für die 30 und die 6 Stadtteile, jeweils bezogen auf die eigene Hälfte.
`zielgroessen()`	Verteilung beider Zielgrößen, getrennt für beide Hälften.
`suchmenge()`	Die Einstellungen für eine Prüfgruppe werden in Wiederholung 0 auf deren Übungsstadtteilen gesucht. Diese Funktion zählt, wie viele Prüfstadtteile der Wiederholungen 1 bis 9 schon zu dieser Suchmenge gehörten.
`zuteilung()` innen · in suchmenge()	Holt für eine Wiederholung die Gruppenzuteilung der 30 Entwicklungsstadtteile aus `v0_aufteilung.py`.
`nachbarschaft()`	Zählt je Wiederholung und Prüfgruppe die Übungsstadtteile, die keine gemeinsame Grenze mit einem Prüfstadtteil haben. Das zeigt, wie viele übrig blieben, wenn man um die Prüfstadtteile einen Puffer legte. Benutzt dieselben Stadtteilgrenzen wie die Aufbereitung.
`_md()` Hilfsfunktion	Wandelt eine Tabelle in eine Texttabelle (Markdown) um.
`bericht()`	Setzt die Tabellen zur Lesefassung `panelprofil.md` zusammen, mit der Zuteilungsregel im Kopf. Hier wird nichts gerechnet.
`main()` Startpunkt	Schreibt die fünf Tabellen und die Lesefassung. Anders als die Modellskripte darf diese Datei beide Hälften sehen, weil sie kein Modell bewertet.

vorpruefung/

v1_baselines.py

Die Messlatte

Rechnet Stufe 1 und Stufe 2 für beide Stränge.

LIEST	SCHREIBT	BENUTZT VON
beide Analysedateien	`results/regression/baselines_*.csv` und `results/klassifikation/baselines_klasse*.csv`	run, v3, m02, m03, m04

Diese Datei legt fest, was ein Vergleichsverfahren mindestens leisten muss. Gerechnet wird über zehn Wiederholungen mal fünf Prüfgruppen, also 50 Läufe je Modell. Weil die Messlatte auf genau denselben Prüfzeilen läuft wie später die Verfahren, lässt sie sich Lauf für Lauf mit ihnen vergleichen. Die beiden Stufe-2-Modelle für den Vergleich stehen hier: Wer sie später braucht, etwa `m02` und `m03` für die Schlussbewertung oder `m04` für die Ablation, holt sie aus dieser Datei.

FUNKTION	WOZU SIE DA IST
MESSEN	
bewerte_regression()	Rechnet die drei Fehlermaße RMSE, MAE und R^2 auf der ursprünglichen Skala der Zielgröße. Auch `v3`, `m04` und die Schlussbewertung in `m02` benutzen diese Funktion.
DIE BEIDEN STUFE-2-MODELLE	
poisson_glm()	Stufe 2 für „Wie viele?“: ein Poisson-Modell, das die Rate je Einwohner schätzt und am Ende mit der Einwohnerzahl hochrechnet. Die Einwohnerzahl geht als fester Bezug ein (Offset). Keine Einstellschraube, deshalb keine Suche.
logit_glm()	Stufe 2 für „Welche Art?“: eine multinomiale logistische Regression. Seltene Einsatzarten werden höher gewichtet, statt Zeilen zu kopieren oder zu löschen. Dieselbe Funktion benutzen `m03_struktur.py` für die Schlussbewertung und `m04_shap.py` für die Ablation.
STRANG MENGE	
regression()	Rechnet für beide Mengen-Zielgrößen Stufe 1 (Durchschnitt der Übungsstadtteile) und Stufe 2 (Poisson-Modell) in allen 50 Läufen.
_zweistufig() Hilfsfunktion	Mittelt erst je Wiederholung über die fünf Prüfgruppen, dann über die zehn Wiederholungen. Berichtet beide Streuungen, maßgeblich ist die über die zehn Wiederholungsmittel, weil die 50 Läufe dieselben 30 Stadtteile in wechselnder Mischung sind.
STRANG STRUKTUR	
klassifikation()	Rechnet Stufe 1 (immer die häufigste Einsatzart) und Stufe 2 (logistische Regression) in allen 50 Läufen und zählt die Warnungen, wenn die Anpassung nicht sauber zu Ende kommt.

FUNKTION	WOZU SIE DA IST
_macro_auroc() Hilfsfunktion	Rechnet die Trennfähigkeit über alle vier Klassen (Macro-AUROC). Fehlt in einer Prüfgruppe eine Klasse, bleibt der Wert leer, statt dass ein erfundener Wert eingesetzt wird.
START	
run() Startpunkt	Führt beide Stränge aus und schreibt die Ergebnisdateien. Schritt 1 von `run.py`.

vorpruefung/

v2_eignung.py

Passen die Verfahren zu den Daten?

Sechs Belege dafür, welche Verfahrensklasse zu welcher Zielgröße passt.

LIEST
beide Analysedateien und die Werte der Messlatte aus v1

SCHREIBT
`results/eignungspruefung/eignungspruefung.md` und `annahmen.csv`

Die Eignungsprüfung rechnet bis auf Beleg 4 auf den 24 Übungsstadtteilen der ersten Prüfgruppe, damit die Prüfstadtteile keine Entscheidung beeinflussen. Sie sammelt sechs Belege, jeder in einer eigenen Funktion. Das Ergebnis ist ein Befundblatt, auf das die Verfahrenswahl in der Arbeit verweist.

Was sie ausdrücklich nicht leistet: Sie entscheidet nicht zwischen Random Forest und XGBoost. Das ist die Forschungsfrage selbst und wird erst in Heft 3 beantwortet.

FUNKTION	WOZU SIE DA IST
AUSGABE	
log() Hilfsfunktion	Gibt eine Zeile auf dem Bildschirm aus und schreibt sie zugleich in den Bericht.
DIE BELEGE 1 BIS 5	
dispersion()	Beleg 1: Streuen die Einsatzzahlen stärker, als ein Poisson-Modell erlaubt? Daraus folgen die Fehlerfunktionen für Random Forest und XGBoost, die zu Zähldaten passen.
linearitaet()	Beleg 2: Vergleicht die Zusammenhangsmaße Pearson und Spearman und passt Ridge einmal auf den Rohwerten und einmal auf log(1+y) an.
spezifikation()	Beleg 3: Prüft mit dem RESET-Test und mit 45 Wechselwirkungen, ob ein gerades Modell Struktur in den Daten übrig lässt.
extrapolation()	Beleg 4: Wie oft liegen Prüfstadtteile außerhalb dessen, was im Training vorkam? Ein hoher Anteil ist eine Grenze der Daten, keine Frage der Verfahrenswahl.
klassifikation()	Beleg 5: Trennen dieselben Merkmale auch die Einsatzarten? Geprüft wird je Merkmal mit dem Kruskal-Wallis-Test.

FUNKTION	WOZU SIE DA IST
ZAHLEN LESBAR SCHREIBEN	
_z() Hilfsfunktion	Schreibt eine Teststatistik mit deutschem Dezimalkomma.
_p() Hilfsfunktion	Schreibt einen p-Wert in deutscher Form, sehr kleine Werte als „< 0,001“.
BELEG 6 UND ERGÄNZUNG	
annahmen()	Beleg 6: Tabelle der Anforderungen je Verfahren, jeweils mit Prüfung, Ergebnis und Folge, wo möglich mit einem formalen Test. Sie führt auch die Anforderungen, die für ein Verfahren gar nicht bestehen, etwa eine Verteilungsannahme bei den Baumverfahren.
Z() innen · in annahmen()	Baut eine Zeile dieser Tabelle, einmal lesbar mit Dezimalkomma und einmal als Zahl für die Weiterverarbeitung.
ridge_rate()	Beleg 2b: Prüft die Umrechnung der Zielgröße so, wie Ridge im Vergleich tatsächlich rechnet: auf der Rate je Einwohner, zurückgerechnet auf die Anzahl.
r2() innen · in ridge_rate()	Rechnet das Bestimmtheitsmaß R^2.
START	
main() Startpunkt	Rechnet die sechs Belege und schreibt Bericht und Tabelle. Schritt 2 von `run.py`.

vorpruefung/

v3_spezifikation.py

Die Gegenprobe

Prüft, ob Krümmung und Wechselwirkungen die Vorhersage für unbekannte Stadtteile verbessern.

LIEST
`regression.parquet` und die Werte der Messlatte aus v1, dazu `klassifikation.parquet` für die Prüfgruppen

SCHREIBT
`results/spezifikation/spezifikation_folds.csv` und `spezifikation_mittel.csv`

Die Datei nimmt das Stufe-2-Poisson-Modell und rechnet es in vier Varianten: gerade, mit Quadraten, mit Wechselwirkungen und mit beidem. Jede Variante läuft im selben Protokoll wie die Messlatte, also in 50 Läufen auf Stadtteilen, die das Modell nicht kennt.

Keine dieser Varianten tritt später im Vergleich an. Sie dienen nur der Frage, ob sich die Krümmung, die `v2_eignung.py` auf den bekannten Stadtteilen findet, auf unbekannte überträgt.

FUNKTION	WOZU SIE DA IST
entwerfe()	Baut die Merkmalstabellen für eine der vier Varianten. Standardisiert wird nur mit Mittelwert und Streuung der Übungsstadtteile, Quadrate und Produkte entstehen erst danach.
ein_lauf()	Passt das Poisson-Modell einmal an und bewertet es auf den Prüfstadtteilen, auf der ursprünglichen Skala. Hält fest, ob die Anpassung zu Ende gekommen ist.
alle_laeufe()	Rechnet 10 Wiederholungen × 5 Prüfgruppen × 4 Varianten, also 200 Anpassungen.
zweistufig()	Mittelt erst je Wiederholung, dann über die Wiederholungen, wie überall im Projekt.
_selbsttest() Hilfsfunktion	Prüft, dass die gerade Variante genau die Stufe-2-Messlatte aus v1 wiederholt. Weicht sie ab, sähe diese Datei andere Daten als v1, und das Skript bricht ab.
run() Startpunkt	Rechnet alle 200 Anpassungen, schreibt die beiden Ergebnisdateien und meldet, wie viele Anpassungen nicht zu Ende gekommen sind. Diese werden gezählt, nicht entfernt.

vorpruefung/

v4_decke.py

Was ist überhaupt erreichbar?

Rechnet zwei Obergrenzen für die Vorhersage der Einsatzart.

LIEST
`klassifikation.parquet`, wenn vorhanden auch die Ergebnisse aus m03

SCHREIBT
`results/klassifikation/decke*.csv` und `decke.md`, mit `holdout` jeweils mit der Endung `_holdout`, ohne `holdout` zusätzlich `klassen_f1.csv`

Gegen den Bestwert 1,0 gehalten wirkt der Macro-F1 des Strukturstrangs schwach. Diese Lesart misst aber an einem Wert, den mit dieser Zielgröße und diesen Merkmalen niemand erreichen kann. `v4_decke.py` rechnet deshalb zwei realistische Obergrenzen.

Decke A: Die überwiegende Einsatzart eines Monats ist selbst ein Zufallsergebnis, wenn zwei Anteile dicht beieinanderliegen. **Decke B:** Fast alle Merkmale gehören zum Stadtteil, nur Jahreszeit und Kriminalitätsindex ändern sich von Monat zu Monat. Ein Modell kann daraus also kaum mehr ableiten als die häufigste Einsatzart des eigenen Stadtteils. Bindend ist die niedrigere Decke B.

FUNKTION	WOZU SIE DA IST
_macro_f1() Hilfsfunktion	Rechnet den Macro-F1 zwischen zwei Reihen von Einsatzarten.

FUNKTION	WOZU SIE DA IST
decke_a()	Obergrenze A: Zieht jeden Stadtteil-Monat aus seinen beobachteten Anteilen neu und misst, wie oft die überwiegende Einsatzart dabei kippt. So gut wäre ein Modell, das die wahren Anteile exakt kennt.
decke_b()	Obergrenze B: Sagt für jeden Stadtteil immer dessen häufigste Einsatzart voraus. Mehr können Merkmale, die zum Stadtteil gehören, kaum hergeben.
marge()	Abstand zwischen dem größten und dem zweitgrößten Anteil je Zeile: Wie knapp fiel die Entscheidung über die überwiegende Einsatzart aus?
_modellwerte() Hilfsfunktion	Holt die Macro-F1-Werte der Verfahren passend zum Lauf: ohne `holdout` die Mittel der Kreuzvalidierung auf den 30 Entwicklungsstadtteilen, mit `holdout` die Schlussbewertung, gehalten gegen Obergrenzen aus allen 36 Stadtteilen.
ausschoepfung()	Rechnet, welchen Teil des Abstands zwischen Mehrheitsklasse und Obergrenze ein Verfahren ausgeschöpft hat. Der Sockel, den schon die Mehrheitsklasse erreicht, zählt dabei nicht als Leistung.
klassen_f1()	Rechnet aus den gespeicherten Vorhersagen von `m03` den F1-Wert je Einsatzart und Verfahren. Er zeigt, worauf der Macro-F1 ruht, denn dieser mittelt vier Klassen mit gleichem Gewicht, und eine davon ist selten. Dazu, wie oft eine Einsatzart vorhergesagt und dabei getroffen wird. Nur ohne `holdout`, weil es für das Hold-out keine gespeicherten Vorhersagen gibt.
_md() Hilfsfunktion	Baut eine Texttabelle von Hand, ohne Zusatzpaket, mit vier Nachkommastellen.
zelle() innen · in _md()	Formatiert eine einzelne Zelle dieser Tabelle.
bericht()	Setzt die Lesefassung `decke.md` zusammen. Hier wird nichts gerechnet.
main() Startpunkt	Rechnet beide Obergrenzen, die Marge, die Ausschöpfung und ohne `holdout` das F1 je Einsatzart und schreibt die Dateien. Ohne `holdout` bleibt das Hold-out gesperrt, mit `holdout` wird es einbezogen.

TEIL 4

In drei Sätzen

> Was man über diesen Ordner wissen muss, wenn man nur eine Minute hat.

1. `v0_aufteilung.py` liefert allen bewertenden Skripten dieselben zehn Mischungen der Prüfgruppen, ohne das Hold-out anzutasten.

2. `v1_baselines.py` legt die Messlatte, `v2_eignung.py` prüft, ob die Verfahren zu den Daten passen, und `v3_spezifikation.py` prüft, ob sich dieser Befund auf unbekannte Stadtteile überträgt.

3. `panelprofil.py` beschreibt die Aufteilung, `v4_decke.py` misst, was beim Strukturstrang überhaupt erreichbar ist.

Weiter geht es in Heft 3 mit dem Ordner `modelle/`: Dort treten die Verfahren gegen diese Messlatte an.